

The Multiple-Telegram-Channels Question

A discussion that started by asking how to fan out, and ended by realizing there was nothing to fan out from. Notes from the session on 30 May 2026.

Where we started

This project, *tg-problem-statements*, exists because Vivek talks to his AI assistant through Telegram voice notes. He walks around, dictates a problem he wants to work on, the assistant captures it as a structured document, and later he copies that document into the correct project folder and lets the assistant work on it autonomously. It is a workaround. The proper way would be for the assistant's app to support voice on every session directly, but that does not work well yet, so Telegram has been the dependable voice channel.

On the list of problems to think about, one entry stood out as special: **multiple-tg-channels-at-once**. The idea was simple. If Telegram could somehow talk to many of Vivek's assistant sessions at the same time, then he would not need this batching project at all. He could speak directly to each session. The entry was explicitly the self-terminator of the whole repository — the problem whose solution would put this project out of business.

What we went looking for

We dispatched a small swarm of research agents. One went to read the official Telegram plugin documentation. Another inspected the plugin's actual source code on GitHub. A third combed blogs, forums, and community projects for anyone who had tried to make many Telegram bots talk to many assistant sessions on the same laptop.

The picture they brought back was mixed. The official plugin technically allows running several sessions if each one has its own bot and its own private state folder — but there is a quiet trap. Every instance of the assistant tool running on the laptop, even the editor extensions, will silently start its own listener and they will end up fighting over messages, with some getting silently dropped. There is even a known bug report on this. Meanwhile, the community-built workarounds, including a popular one called *shaike1/relay*, take a completely different approach — they pretend to be a human typing into a terminal pane. Clever, but it would break every custom rule Vivek has built up: the voice transcription pipeline, the food-photo detector, the WhatsApp generalizations. All of those rules listen for a very specific message shape that the community relay does not produce.

So we had a real choice on our hands: pay an ongoing tax of careful configuration with the official multi-bot setup, or pay a one-time migration tax to adopt a community tool and lose the existing rules. We were about to weigh them when Vivek pulled the conversation up to a higher altitude.

The reframing through Amdahl's Law

He mentioned Amdahl's Law. Gene Amdahl wrote a paper in 1967 making a deflating point about parallel computing: every workload has portions that are inherently *serial* — they cannot be split up no matter how many machines you throw at them. And as you scale up, the serial portion ends up dominating the total time. Parallel computing, he argued, is not a free lunch. It has hard mathematical ceilings.

Applied to Vivek's workflow, the realization was uncomfortable but clean: **he is the inherently serial part**. The bottleneck is not the Telegram bot, not the listener architecture, not anything Anthropic might ship in a future release. It is one brain, one mouth, one stream of attention. Dictating a problem happens at the speed of speech, through one person, one statement at a time. No infrastructure can remove that.

That is when the question started to dissolve.

The clean conclusion

The workflow has two phases, and they have very different shapes. The **setup phase** — listing the problems, sorting them into “straightforward” versus “exploratory”, and dictating each statement — is unavoidably serial because Vivek is doing it. The **execution phase** — the assistant running autonomously on a well-formed statement — is genuinely parallel, and importantly, it needs no voice at all. The agent is just working.

Once you see that split, the multi-channel question collapses. In the setup phase, more Telegram channels would not help, because Vivek can only voice-talk to one at a time anyway — the other channels would sit idle. In the execution phase, voice is not needed at all, so the channels would be irrelevant. The original problem statement was what we called a *category error* — a fix for a problem that does not structurally exist.

We then compared the current setup to an imagined ideal world — one where every assistant session has first-class voice support and a clean cross-session view on the phone. The surprise: the two are almost identical. The setup phase looks the same in both (one voice, one brain). The execution phase looks the same in both (autonomous, no voice needed). The **only** difference is that in the current setup, Vivek has to copy each finished problem statement from this project folder into its target project folder before launching the agent. One file copy per problem. That is the entire structural gap.

So the conclusion turned the project on its head. This is not a temporary workaround waiting to be obsoleted. It is, under the constraints of having a single human at the helm, the right architecture. The only knob worth turning is Vivek's own discipline: stay within one topic area per session. The hard friction in today's session was not technical — it was the cognitive cost of jumping from research-paper problems to model-training problems and back. Same domain flows. Cross-domain stalls. That is the operational lesson.

Do not build more Telegram channels. Do not adopt the community relays. Do not wait for Anthropic to ship anything special. Keep doing this, one domain at a time, and copy the file when you are done.